STELLAR WINGS C

Shoot'em up pour GX4000 et CPC+, écrit en C (SDCC) et en assembleur Z80

Documentation complète du code — projet GX Studio 0.5.1



L'écran titre : logo en dégradé « copper », défilé d'aliens, musique

STELLAR WINGS est un shoot'em up original pour la GX4000 et les CPC+, dans l'esprit de Space Hawks et de Space Invaders. Une formation de 24 aliens descend en se balançant, certains plongent en piqué vers le vaisseau, un vaisseau mère attaque toutes les quatre vagues. On joue à un ou deux joueurs (en alternance).

Ce projet est la **version C** du jeu. La version d'origine est écrite en assembleur. Le jeu est identique : mêmes règles, mêmes vitesses, mêmes sprites, mêmes sons, même cartouche de 48 Ko. La logique est écrite en C et compilée par **SDCC**. L'affichage passe par la bibliothèque Z80 de GX Studio. Deux routines très souvent exécutées restent en **assembleur** : le **profil** d'exécution du studio a montré qu'elles empêchaient le jeu en C de tenir 50 images par seconde.

Ce document décrit tout le code : l'organisation du projet, le matériel utilisé, chaque fichier et chaque fonction, les routines en assembleur, les mesures de performance, les ressources et la mise au point. Le **tutoriel** (Stellar_Wings_C_tutoriel.pdf) montre comment construire le jeu pas à pas.

Chapitre	Contenu
1. Le jeu	règles, commandes, déroulement
2. Le projet dans GX Studio	ouvrir, compiler, lancer ; les fichiers ; la cartouche
3. Organisation du code	les fichiers C et assembleur, stellar.h, les types
4. Le matériel et la bibliothèque	mémoire, sprites, rasters, multiplexage, son DMA, API gx.h
5 à 8. Les fichiers C	main.c, play.c, aliens.c, fx.c, fonction par fonction

9. Les routines en assembleur	stars.asm, bands.asm, appel depuis le C
10. Performances	le profil, les mesures, pourquoi deux routines en assembleur
11. Les ressources	sprites, palettes, sons, logo
12. Mise au point	options, points d'arrêt, variables
13. Comparaison avec la version assembleur	ce qui change, ce qui ne change pas
Annexe	index des fonctions

1. Le jeu



Une vague d'aliens (à gauche) et le vaisseau mère (vague 4)

Commande	Effet
Écran titre : TIR 1 / TIR 2	partie à un joueur / à deux joueurs (chacun son tour)
Gauche, droite	déplace le vaisseau
Tir (maintenu)	tir automatique, un tir toutes les 10 images
Manette 1 ou 2	sur CPC+ aussi les flèches et Espace / COPY

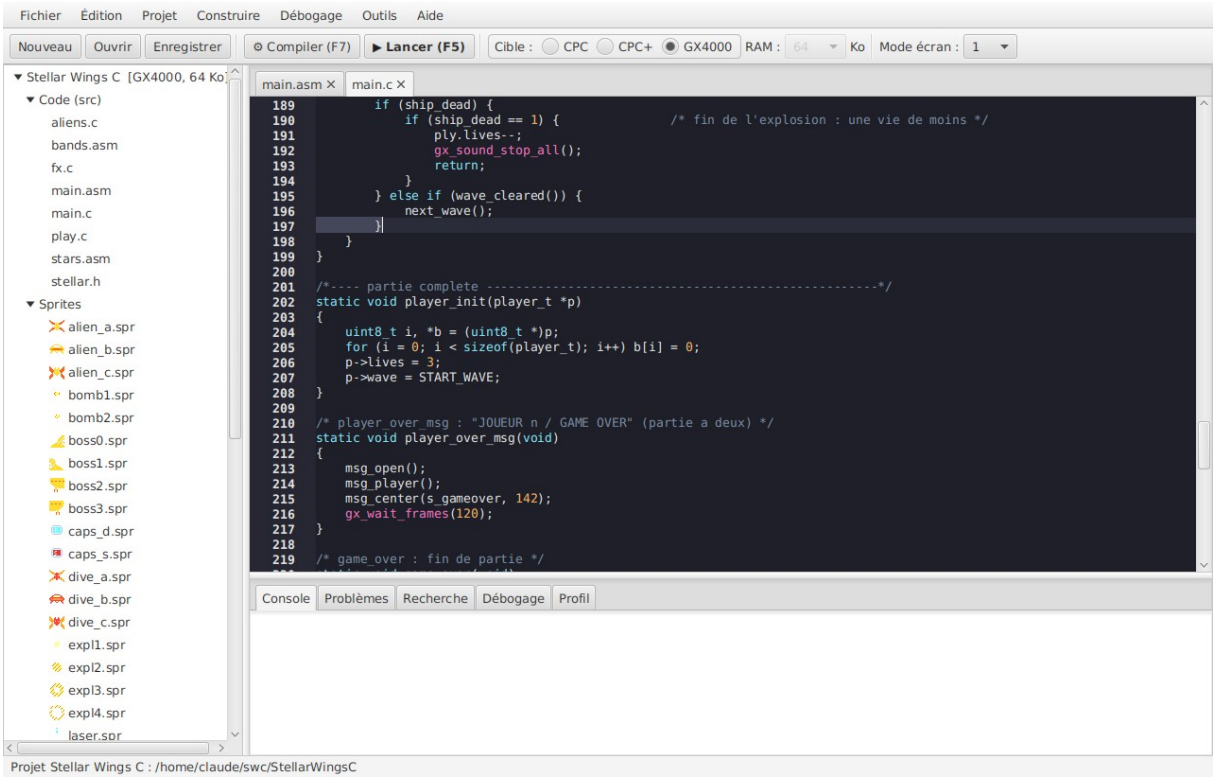
- La formation compte **4 rangées de 6 aliens**. Elle se balance de gauche à droite et descend de 4 lignes à chaque demi-tour. Elle accélère quand elle perd des aliens ; le dernier fonce. Si elle atteint le vaisseau, c'est l'invasion : une vie de moins.
- Des aliens quittent la formation **en piqué** (jusqu'à trois à la fois) : ils décollent, se dirigent vers le vaisseau en lâchant des bombes, repassent par le haut de l'écran et reprennent leur place.
- Points par alien : 50, 40, 30, 20 selon la rangée (de haut en bas), le double en piqué. Vaisseau mère : 10 points par tir, 5 000 points quand il est détruit. **Vie supplémentaire à 10 000 points.**

- Un plongeur abattu lâche une fois sur trois une **capsule** : **D** = tir double, **S** = vaisseau plus rapide. On les garde jusqu'à la perte d'une vie.
- Toutes les **4 vagues**, un **vaisseau mère** (128 x 64 pixels) remplace la formation. Ses points de vie augmentent avec la vague (20 + 2 x vague, 60 au plus) ; il accélère quand il est blessé.

Le vaisseau est perdu s'il est touché par une bombe, un plongeur ou la formation. À deux joueurs, chaque perte de vie passe la main à l'autre joueur, qui reprend sa partie exactement où il l'avait laissée (score, vague, aliens restants, capsules perdues).

2. Le projet dans GX Studio

Fichier > Ouvrir un projet puis **projet.gxs**, et **F5** : le studio convertit les ressources, compile le C avec SDCC, assemble le tout avec rasm et lance la cartouche dans Caprice32. Il faut GX Studio **0.5.1** (avec SDCC installé : sudo apt install sdcc) et le Caprice32 corrigé fourni avec le studio. La cartouche **build/stellarw.cpr** se copie telle quelle sur la PicoGX.



Le projet ouvert dans GX Studio : à gauche les fichiers C et assembleur, les sprites, les sons

Dossier / fichier	Contenu
projet.gxs	nom, cible GX4000, 64 Ko, mode 1, fichier principal src/main.asm, définitions
src/main.c, play.c, aliens.c, fx.c	le jeu en C (chapitres 5 à 8)
src/stellar.h	types, constantes, variables et fonctions partagées
src/stars.asm, src/bands.asm	les deux routines en assembleur appelées par le C (chapitre 9)
src/main.asm	assemble la cartouche : bibliothèque, liaison C, code C traduit, ressources
assets/sprites (20 fichiers .spr)	aliens, plongeurs, vaisseau, tirs, bombes, explosions, capsules, vaisseau mère
assets/palettes	ecran.pal (encres 0-3), sprites.pal (couleurs 1-15 des sprites)
assets/sons (10 fichiers .snd)	musique du titre, jingles, marche de la formation, bruitages
assets/images/logo.png	le logo STELLAR WINGS, converti en mode 1

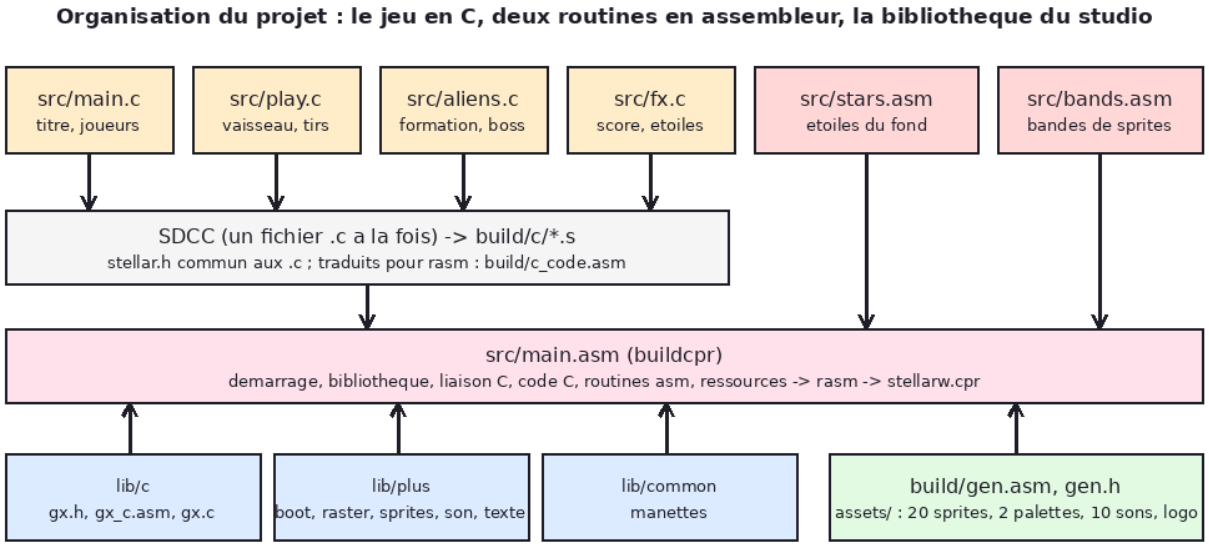
lib/	bibliothèque de GX Studio : lib/plus (CPC+), lib/common, lib/c (pour le C)
build/	fichiers produits par le studio : gen.asm, gen.h, c_code.asm, sprites.bin, stellarw.cpr…
doc/	cette documentation et le tutoriel



La cartouche : 3 pages de 16 Ko

La page 0 est la ROM basse, en &0000-&3FFF : elle contient tout le code, et l'écran de présentation de GX Studio (logo FANOU 1972, 3 secondes au démarrage, environ 2,6 Ko : la page 0 est remplie à 96 %). La page 1 contient les images des sprites, que la bibliothèque recopie dans l'ASIC quand un sprite change d'image. La page 2 contient la police et le logo ; elle est visible en &C000 en lecture, alors que les écritures à ces adresses vont dans l'écran.

3. Organisation du code



Les fichiers du projet et la construction de la cartouche

Le jeu est découpé comme la version en assembleur, un fichier par grand thème. Chaque fichier C inclut **stellar.h**, qui inclut **gx.h** (l'API de la bibliothèque du studio), qui inclut **build/gen.h** (les ressources : numéros d'images F_..., sons, palettes, logo).

Fichier	Rôle	Fonctions principales
main.c	démarrage, écran titre, parties, joueurs	main, title_screen, game, play_turn, next_wave, game_over
play.c	une image de jeu, le vaisseau, ses tirs, la capsule	play_frame, turn_reset, ship_update, bullet_one, caps_update
aliens.c	formation, tables raster, plongeurs, bombes, vaisseau mère	form_update, build_raster, diver_one, hit_formation, boss_update

fx.c	hasard, score, panneau, messages, étoiles, explosions	rand8, add_score, hud_update, msg_center, stars_init, kill_ship
stars.asm	les 40 étoiles du fond (assembleur)	_stars_update
bands.asm	une bande de sprites multiplexés (assembleur)	_band_row

3.1 stellar.h : constantes et types

```
/*---- dimensions ---------------------------------------------------------*/
#define SHIP_Y      172         /* ligne du vaisseau */
#define LOGO_Y      16          /* ligne du logo sur l'ecran titre */
#define MSG_TOP     124         /* zone des messages : lignes 124 a 151 */
#define MSG_LINES   28
#define COL_PITCH   48          /* ecart entre colonnes de la formation (pixels mode 2) */
#define ROW_PITCH   24          /* ecart entre rangees (lignes) */
#define NDIVERS     3           /* plongeurs simultanes au maximum */
#define NSTARS      40          /* etoiles du fond */

/*---- mise au point (Projet > Proprietes : START_WAVE=4, GOD_MODE=1) ------*/
#ifndef START_WAVE
#define START_WAVE  1           /* vague de depart */
#endif
#ifndef GOD_MODE
#define GOD_MODE    0           /* 1 = vaisseau invincible */
#endif

/*---- sprites materiels --------------------------------------------------
 *  0-5   formation (multiplexee : 4 rangees par interruptions raster) ou boss (0-3)
 *  6     vaisseau        7-8  tirs du joueur     9-10 bombes
 *  11    capsule         12-14 plongeurs         15   explosion          */
#define SPR_SHIP    6
#define SPR_BULLET  7
#define SPR_BOMB    9
#define SPR_CAPS    11
#define SPR_DIVER   12
#define SPR_EXPL    15

/*---- types --------------------------------------------------------------*/
typedef struct {                /* un joueur (deux blocs : partie a deux en alternance) */
    uint16_t score;             /* en dizaines de points (affiche avec un 0 de plus) */
    uint8_t  lives;
    uint8_t  wave;              /* numero de vague (1, 2...) */
    uint8_t  alive[4];          /* aliens vivants : rangee r, bit c = colonne c */
    uint8_t  power;             /* bit 0 : tir double (capsule D), bit 1 : vitesse (S) */
    uint8_t  over;              /* 1 = partie terminee pour ce joueur */
    uint8_t  bhp;               /* points de vie du vaisseau mere */
} player_t;

typedef struct {                /* tir du joueur ou bombe */
    uint8_t on;
    int16_t x;                  /* pixels mode 2 */
    uint8_t y;
} shot_t;

typedef struct {                /* alien en pique */
    uint8_t state;              /* 0 libre, 1 decollage, 2 attaque, 3 retour */
    uint8_t row, col;           /* sa place dans la formation */
    int16_t x, y;               /* y signe : il repasse par le haut de l'ecran */
    int8_t  vx;
    uint8_t timer, bomb;
} diver_t;
```

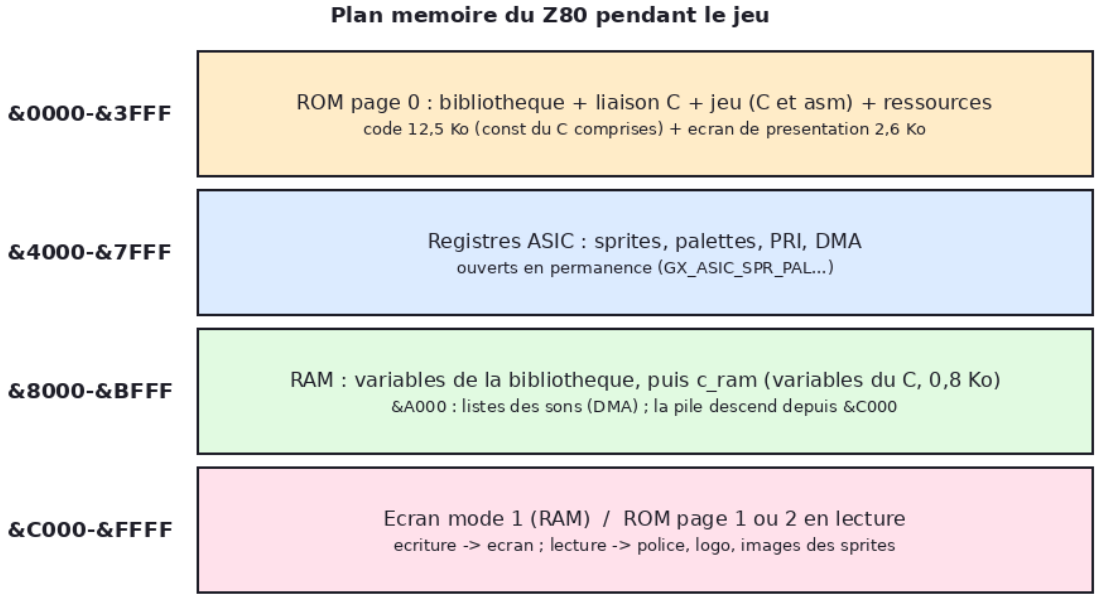
Les variables partagées entre fichiers sont déclarées `extern` dans stellar.h et définies dans un seul fichier. Celles qui ne servent qu'à un fichier sont `static` dans ce fichier : par exemple la position de la formation (fx, fy) n'existe que dans aliens.c. SDCC range toutes les variables en RAM, dans le bloc **c_ram** réservé par main.asm (764 octets).

3.2 Choix des types

- **uint8_t** partout où c'est possible : le Z80 calcule sur 8 bits ; un `int` (16 bits) coûte deux à trois fois plus.
- **int16_t** pour les abscisses : les sprites se placent en pixels « mode 2 », de 0 à 639, et la formation peut déborder à gauche (fx négatif).
- **int8_t** pour les vitesses (ball_dx des plongeurs : -4 à 4).
- Le **score** est un `uint16_t` compté en **dizaines de points** : tous les points du jeu sont des multiples de 10, et 65 535 dizaines suffisent (655 350 points). L'affichage écrit 5 chiffres puis un 0. La version assembleur utilisait du BCD ; en C, un entier est plus simple et évite les divisions sur 32 bits.
- Pas de nombres à virgule : la formation avance en **quarts de pixel** (fsub).

4. Le matériel et la bibliothèque

4.1 La mémoire



Plan mémoire pendant le jeu

La bibliothèque laisse la page des registres de l'ASIC ouverte en &4000-&7FFF pendant tout le jeu. Le C peut donc écrire directement dans les palettes de l'ASIC avec `GX_ASIC_PAL` et `GX_ASIC_SPR_PAL` (gx.h). Les variables commencent en &8000 : d'abord celles de la bibliothèque (déclarées par VAR dans main.asm), puis le bloc c_ram du C. Les listes des sons sont recopiées en &A000, car le DMA de l'ASIC lit la RAM. La pile descend depuis &C000.

4.2 Les 16 sprites matériels

Sprites	Utilisation	Géré par
0-5	la formation : 6 aliens par rangée, réutilisés pour les 4 rangées (multiplexage) ; ou les 4 quarts du vaisseau mère	bandes raster (build_raster)
6	vaisseau du joueur	ship_update
7-8	tirs du joueur (le 8 avec la capsule D)	bullet_one
9-10	bombes	bomb_one
11	capsule	caps_update

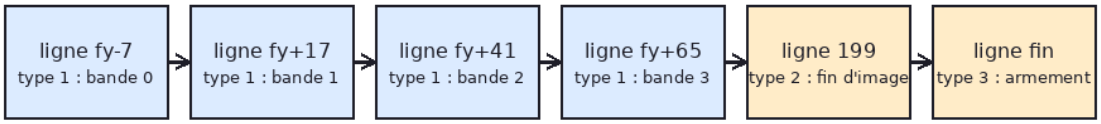
12-14	plongeurs	diver_one
15	explosion	expl_update

Un sprite fait 16 x 16 pixels de 15 couleurs. Le jeu les affiche avec un zoom x2 en largeur (ZOOM_X2Y1) : 32 pixels mode 2 de large. Les sprites 6 à 15 se règlent avec `gx_spr_set(n, x, y, image, zoom)`, qui écrit dans une copie en RAM (l'ombre). À la fin de chaque image, `gx_spr_update_from(6)` recopie les sprites 6 à 15 dans l'ASIC et charge les images qui ont changé. Les sprites 0 à 5 ne sont pas recopiés : ils sont gérés par les interruptions raster.

4.3 Les interruptions raster et le multiplexage

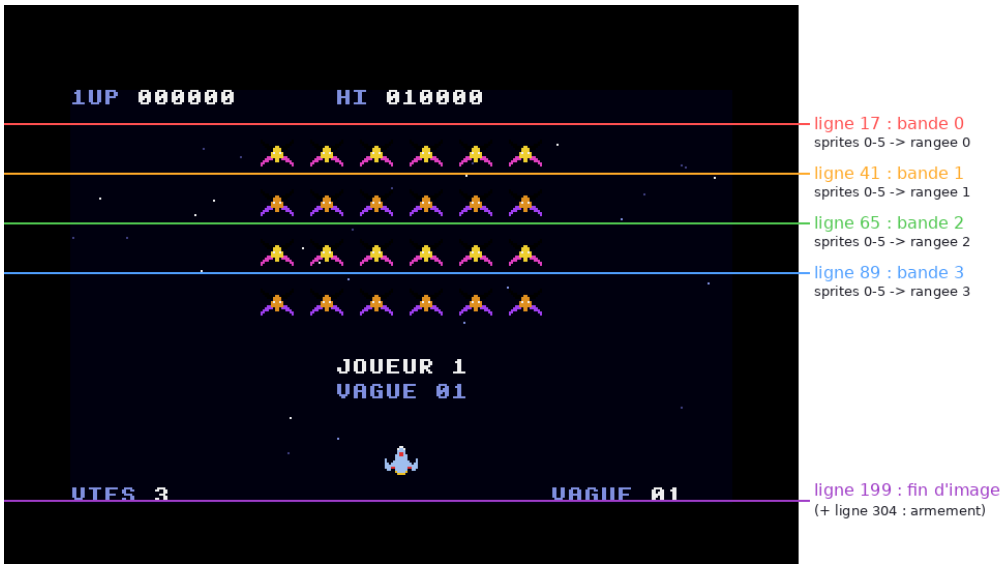
Le registre PRI de l'ASIC déclenche une interruption quand l'écran est à une ligne donnée. La bibliothèque (lib/plus/raster.asm) enchaîne ces interruptions d'après une **table raster**. La table commence par la ligne de la première interruption, puis contient des entrées de 4 octets : un type, une donnée et la ligne de l'interruption suivante.

Une table raster (double tampon) : chaque interruption programme la suivante



type 1 : recopie 6 x (X, Y, zoom) + 4 couleurs dans l'ASIC (bloc de 38 octets : band_row)
type 2 : frame_flag = 1 (gx_wait_frame revient), PRI = ligne d'armement mesuree (pri_arm)
type 3 : bascule sur la table publiee par gx_raster_queue, PRI = sa 1re ligne
type 0 (ecran titre) : change la couleur de l'encre 3 => degrade "copper"

La table raster du jeu

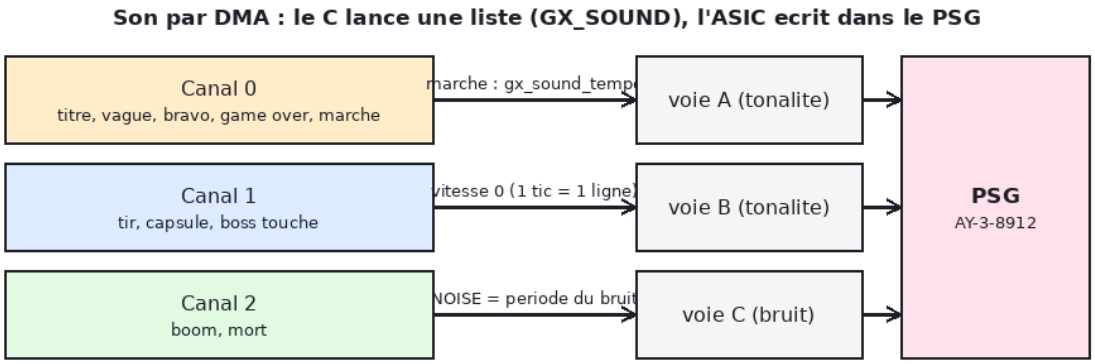


Multiplexage : les sprites 0-5 repositionnés juste avant chaque rangée

Pendant le jeu, la table contient une entrée **type 1** (bande de sprites) par rangée, 7 lignes avant elle. L'interruption recopie un bloc de 38 octets dans l'ASIC : X, Y et zoom des sprites 0 à 5, puis les couleurs 1 à 4 des sprites. Les six sprites montrent ainsi successivement les quatre rangées, chacune avec ses couleurs. Le battement d'ailes est un simple jeu de palette : la couleur 3 (ailes hautes) ou la couleur 4 (ailes basses) passe au noir une image sur deux.

Le jeu calcule la table de l'image suivante pendant que l'interruption lit la table en cours : il y a deux tables (**double tampon**, rtab[2] et bands[2] dans aliens.c). `gx_raster_queue` publie la nouvelle table. L'interruption d'armement (type 3) bascule dessus à la fin de l'image : on ne modifie jamais une table en cours de lecture.

4.4 Le son par DMA



Les trois canaux DMA

Chaque fichier .snd devient une liste d'instructions DMA (build/gen.asm). `GX_SOUND(NOM)` la lance sur son canal, avec sa vitesse. Le son joue ensuite tout seul, sans le Z80. La **marche** de la formation est jouée en boucle sur le canal 0. Sa vitesse est changée en cours de lecture par `gx_sound_tempo` : le tempo accélère quand la formation fond, comme dans Space Invaders.

4.5 Les fonctions de la bibliothèque utilisées

Fonction (gx.h)	Utilisation dans le jeu
gx_init, gx_wait_frame, gx_wait_frames	démarrage ; synchronisation sur la fin d'image (ligne 199)
gx_read_joy ; joy, joy_new, frame_count	manettes ; compteur d'images (animations, hasard)
gx_clear_screen, gx_clear_lines, gx_palette, gx_sprite_palette	effacements, palettes (pal_screen, pal_title, pal_sprites)
gx_ink, gx_text, gx_text_num, gx_blit	textes, nombres, logo (IMG_LOGO)
gx_spr_set, gx_spr_hide, gx_spr_hide_all, gx_spr_update_from ; spr_want	sprites 6-15 ; images des sprites 0-5
gx_raster_start, gx_raster_queue ; raster_idle, RASTER_ARM_LINE, GX_BAND_SIZE	tables raster
GX_SOUND, gx_sound, gx_sound_stop, gx_sound_stop_all, gx_sound_tempo, gx_psg	sons
GX_ASIC_SPR_PAL	couleurs des sprites (battement d'ailes, titre)

Toutes ces fonctions sont des petites routines de liaison (lib/c/gx_c.asm) : elles lisent leurs paramètres sur la pile et appellent la routine assembleur de la bibliothèque. Leur coût est faible devant le travail de la routine elle-même.

5. main.c : titre, parties et joueurs

`main()` est appelée par src/main.asm après le démarrage de la console. Elle initialise la bibliothèque et le hasard, puis enchaîne sans fin l'écran titre et une partie.

```
/*---- main : appele en boucle par src/main.asm ----------------------------*/
void main(void)
{
    gx_init();                                  /* son (DMA), sprites */
    rnd = 0xACE1;
    hiscore = 1000;                             /* record initial : 10 000 points */
    for (;;) {
        title_screen();
        game();
    }
}
```

5.1 L'écran titre

`title_screen` efface tout (screen_reset), affiche le logo avec `gx_blit`, les textes centrés et le record, et lance la musique (GX_SOUND(TITRE)). Ensuite, à chaque image, il construit une table raster qui change 16 fois la couleur de l'encre 3 sur le logo, avec un décalage à chaque image : le dégradé défile (effet « copper »). Il fait aussi se balancer six aliens. Le premier appui sur TIR 1 ou TIR 2 choisit le nombre de joueurs. Le moment de l'appui est mélangé à la graine du hasard, pour que deux parties ne se ressemblent pas.

```
static void title_raster(void)
{
    uint8_t *t = next_tab();
    uint8_t e, line = LOGO_Y + 4, i;
    *t++ = LOGO_Y;
    e = (frame_count >> 1) % NB_COPPER;
    for (i = 0; i < 16; i++) {
        *t++ = 0;                                   /* type 0 : encre 3 */
        *t++ = (uint8_t)copper[e];
        *t++ = copper[e] >> 8;
        *t++ = line;
        line += 4;
        if (++e == NB_COPPER) e = 0;
    }
    *t++ = 0;                                       /* puis l'encre 3 redevient fixe (textes) */
    *t++ = (uint8_t)pal_title[3];
    *t++ = pal_title[3] >> 8;
    *t++ = 199;
    tab_tail(t);
}
```

5.2 Une partie, deux joueurs

Chaque joueur a un bloc **player_t** (pblk[0], pblk[1]). Le joueur en cours est recopié dans **ply**, la seule copie que le reste du jeu utilise. À la fin du tour, ply est rangé dans son bloc. À deux joueurs, la main passe à l'autre joueur si sa partie n'est pas terminée.

```
static void game(void)
{
    player_init(&pblk[0]);
    player_init(&pblk[1]);
    if (nplayers != 2) pblk[1].over = 1;        /* joueur 2 absent */
    cur_player = 0;
    for (;;) {
        ply = pblk[cur_player];                 /* copie de travail */
        play_turn();
        if (!ply.lives) {
            ply.over = 1;
            update_hi();
            if (nplayers == 2) player_over_msg();
        }
        pblk[cur_player] = ply;
        if (nplayers == 2 && !pblk[cur_player ^ 1].over) {
            cur_player ^= 1;                    /* au tour de l'autre joueur */
        } else if (ply.over) {
            game_over();                        /* plus personne : fin de partie */
            return;
        }
    }
}
```

5.3 Un tour de jeu

`play_turn` prépare l'écran (palettes, étoiles, panneau), la vague et les positions de départ. Il affiche « JOUEUR n / VAGUE nn » pendant 110 images de jeu figé (frozen = 1 : tout s'anime mais rien ne bouge), puis lance la marche et joue des images jusqu'à la fin de l'explosion du vaisseau. Quand une vague est terminée, `next_wave` affiche BRAVO, passe à la vague suivante et relance.

```
static void play_turn(void)
{
    screen_reset();
    gx_palette(pal_screen);
    gx_sprite_palette(pal_sprites);
    stars_init();
    hud_draw();
    wave_prepare();
    turn_reset();
    frozen = 1;
    msg_open();
    msg_player();
    msg_wave();
    GX_SOUND(VAGUE);
    play_frames(110);
    msg_close();
    frozen = 0;
    march_start();
    for (;;) {
        play_frame();
        if (ship_dead) {
            if (ship_dead == 1) {               /* fin de l'explosion : une vie de moins */
                ply.lives--;
                gx_sound_stop_all();
                return;
            }
        } else if (wave_cleared()) {
            next_wave();
        }
    }
}
```

Fonction	Rôle
screen_reset	coupe sons, sprites et rasters, palette noire, efface l'écran
title_raster, title_parade, title_screen	écran titre (5.1)
msg_player, msg_wave	« JOUEUR n », « VAGUE nn » ou « ALERTE : VAISSEAU MERE ! »
play_frames(n)	n images de jeu (pendant les messages)
next_wave	vague terminée : BRAVO, vague suivante
play_turn	un tour, jusqu'à la perte d'une vie
player_init, player_over_msg, game_over, game	partie complète, fin de partie

6. play.c : une image de jeu

`play_frame` est le cœur du jeu : elle est appelée 50 fois par seconde et fait avancer tout ce qui bouge. L'ordre compte. Les sprites sont recopiés juste après la fin d'image (`gx_wait_frame`), quand rien n'est affiché. La table raster de l'image suivante est construite en dernier, quand toutes les positions sont connues.

```
void play_frame(void)
{
    dbg_loops++;
    gx_wait_frame();                    /* ligne 199 : bord bas, rien n'est affiche */
    gx_spr_update_from(SPR_SHIP);       /* sprites 6-15 (les 0-5 sont multiplexes) */
    /* battement d'ailes des plongeurs : couleur 7 ou 8 des sprites au noir */
    if (wing) {
        GX_ASIC_SPR_PAL[6] = 0;
        GX_ASIC_SPR_PAL[7] = ((const uint16_t *)pal_sprites)[7];
    } else {
        GX_ASIC_SPR_PAL[6] = ((const uint16_t *)pal_sprites)[6];
        GX_ASIC_SPR_PAL[7] = 0;
    }
    gx_read_joy();
    stars_update();
    ship_update();
```

```
    bullets_update();
    if (is_boss) {
        boss_update();
    } else {
        form_update();
        divers_update();
    }
    bombs_update();
    caps_update();
    expl_update();
    build_raster();                     /* table raster de l'image suivante */
    if (hud_dirty) hud_update();
}
```

6.1 Le vaisseau

Le vaisseau avance de 4 pixels mode 2 par image (7 avec la capsule S) et reste entre 8 et 600. Le bouton de tir maintenu tire automatiquement, avec 10 images d'attente entre deux tirs. Avec la capsule D, deux tirs partent ensemble, 10 pixels à gauche et à droite. Quand le vaisseau est touché, ship_dead compte 120 images. Pendant les 50 premières, une explosion repart toutes les 16 images à sa place. Quand ship_dead arrive à 1, play_turn retire une vie.

```
static void ship_update(void)
{
    int16_t step;
    if (ship_dead) {                    /* explosions en chaine, puis plus rien */
        if (ship_dead == 1) return;
        ship_dead--;
        if (ship_dead >= 70 && !(ship_dead & 15)) expl_quiet(ship_x, SHIP_Y);
        return;
    }
    if (!frozen) {
        step = (ply.power & 2) ? 7 : 4; /* capsule S : vaisseau plus rapide */
        if (joy & JOY_LEFT) {
            ship_x -= step;
            if (ship_x < 8) ship_x = 8;
        }
        if (joy & JOY_RIGHT) {
            ship_x += step;
            if (ship_x > 600) ship_x = 600;
        }
        if (fire_cool) {
            fire_cool--;
        } else if (joy & JOY_FIRE) {    /* bouton maintenu = tir automatique */
            uint8_t fired = 0;
            if (ply.power & 1) {        /* capsule D : deux tirs paralleles */
                if (!bullets[0].on && !bullets[1].on) {
                    fire_bullet(&bullets[0], -10);
                    fire_bullet(&bullets[1], 10);
                    fired = 1;
                }
            } else if (!bullets[0].on) {
                fire_bullet(&bullets[0], 0);
                fired = 1;
            }
            if (fired) {
                fire_cool = 10;
                GX_SOUND(TIR);          /* canal DMA 1 (voie B) */
            }
        }
    }
    gx_spr_set(SPR_SHIP, ship_x, SHIP_Y, F_SHIP, ZOOM_X2Y1);
}
```

6.2 Les tirs et la capsule

Un tir monte de 7 lignes par image. `bullet_hits` teste la pointe du laser : le centre du sprite (x + 16), à sa ligne. Pendant une vague normale, le test porte d'abord sur les plongeurs, puis sur la formation ; pendant le vaisseau mère, sur le vaisseau mère seul. La capsule tombe d'une ligne par image ; elle est attrapée si elle passe à moins de 24 pixels du vaisseau, entre les lignes 158 et

183.

```
static void bullet_one(shot_t *b, uint8_t spr)
{
    if (b->on) {
        if (b->y < 7 + 10) {
            b->on = 0;                  /* sorti par le haut */
        } else {
            b->y -= 7;
            bullet_hits(b);
            if (b->on) {
                gx_spr_set(spr, b->x, b->y, F_LASER, ZOOM_X2Y1);
                return;
            }
        }
    }
    gx_spr_hide(spr);
}

static void bullets_update(void)
{
    bullet_one(&bullets[0], SPR_BULLET);
```

7. aliens.c : formation, plongeurs, bombes, vaisseau mère

7.1 La formation

Les aliens vivants sont rangés dans **ply.alive[4]** : un octet par rangée, bit c = colonne c. Un alien en piqué garde son bit dans alive, mais son bit est aussi posé dans **diving[4]** : il n'est plus dessiné dans la formation. `count_alive` recalcule le nombre d'aliens, les colonnes extrêmes (lcol, rcol) et la dernière rangée occupée (brow) après chaque destruction.

```
void count_alive(void)
{
    uint8_t r, a, n = 0, cols = 0;
    brow = 0;
    for (r = 0; r < 4; r++) {
        a = ply.alive[r];
        if (a) {
            cols |= a;
            brow = r;
            for (; a; a >>= 1) n += a & 1;
        }
    }
    alive_count = n;
    if (!cols) return;
    for (lcol = 0; !(cols & bits[lcol]); lcol++) ;
    for (rcol = 5; !(cols & bits[rcol]); rcol--) ;
}
```

`form_update` fait avancer la formation en quarts de pixel. La vitesse dépend du nombre d'aliens restants et de la vague. Quand la colonne extrême touche un bord, la formation repart dans l'autre sens et descend de 4 lignes (jusqu'à la ligne 148). Si la dernière rangée occupée arrive à la hauteur du vaisseau, c'est l'invasion : `kill_ship`.

```
void form_update(void)
{
    uint8_t v, s, edge;
    int16_t ex;
    wing = (frame_count >> 3) & 1;
    if (march_on) {
        v = march_tempo_val();
        if (v != last_tempo) {
            last_tempo = v;
            gx_sound_tempo(v);
        }
    }
    if (frozen || !alive_count) return;
    v = ((28 - alive_count) >> 2) + 3;          /* vitesse en 1/4 de pixel par image */
    s = ply.wave >> 1;
```

```
    v += s < 7 ? s : 6;
    if (alive_count == 1) v += 8;               /* le dernier alien fonce */
    s = fsub + v;
    fsub = s & 3;
    v = s >> 2;
    if (!fdir) {
        fx += v;
        ex = fx + rcol * COL_PITCH;             /* X de la colonne la plus a droite */
        if (ex < 640 - 16 - 32) return;
    } else {
        fx -= v;
        ex = fx + lcol * COL_PITCH;             /* X de la colonne la plus a gauche */
        if (ex >= 16) return;
    }
    fdir ^= 1;                                  /* demi-tour... */
    if (fy < 148) fy += 4;                      /* ...et la formation descend d'un cran */
    edge = brow * ROW_PITCH + fy + 16;
    if (edge >= 164) kill_ship();               /* invasion : la formation atteint le
vaisseau */
}
```

7.2 La table raster de chaque image

`build_raster` écrit la table de l'image suivante dans le tampon libre (`next_tab`). Pour chaque rangée, elle ajoute une entrée « bande » et remplit le bloc de 38 octets avec `band_row`, en assembleur (chapitre 9). Pour le vaisseau mère, `build_boss` utilise une seule bande : les sprites 0 à 3 en zoom x4 / x2 forment un vaisseau de 128 x 64 pixels. Ils deviennent blancs quand il est touché (bflash).

```
void build_raster(void)
{
    uint8_t *t = next_tab(), *b = br_blk;
    uint8_t r, y = fy;
    if (is_boss) {
        build_boss(t);
        return;
    }
    *t++ = fy - 7;                              /* 1re interruption : 7 lignes avant la
rangee 0 */
    for (r = 0; r < 4; r++) {
        *t++ = 1;                               /* entree : bande de sprites */
        *t++ = (uint8_t)(uint16_t)b;
        *t++ = (uint16_t)b >> 8;
        *t++ = r == 3 ? 199 : y + ROW_PITCH - 7;
        band_row(b, fx, y, ply.alive[r] & ~diving[r], rowpal + r * 4, wing);
        b += GX_BAND_SIZE;
        y += ROW_PITCH;
    }
    tab_tail(t);
}
```

```
void tab_tail(uint8_t *p)
{
    p[0] = 2; p[1] = 0; p[2] = 0; p[3] = RASTER_ARM_LINE;
    p[4] = 3; p[5] = 0; p[6] = 0; p[7] = 0;
    gx_raster_queue(cur_tab);                   /* prise en compte a la fin de l'image */
}
```

7.3 Les plongeurs

Trois plongeurs au plus (sprites 12 à 14). `div_launch` en lance un à intervalles réguliers. Le délai vaut 110 - 8 x vague images, 30 au minimum, plus un hasard. Le nombre de plongeurs simultanés vaut 1 + vague / 2. Le plongeur choisi est l'alien le plus bas d'une colonne tirée au hasard. Chaque plongeur suit une petite **machine à états** :

État	Comportement	Passage à l'état suivant
1 décollage	monte de 2 lignes par image et s'écarte vers le bord le plus proche	après 12 images

2 attaque	descend de 2 à 4 lignes par image ; toutes les 4 images sa vitesse horizontale se rapproche du vaisseau (-4 à 4) ; lâche des bombes au-dessus de la ligne 140	sorti en bas (ligne 200) : réapparaît à la ligne -24
3 retour	suit sa colonne dans la formation et descend de 2 lignes	arrivé à sa place : reprend sa place (bit diving effacé)
0 libre	sprite caché	lancé par div_launch

```
static void diver_one(diver_t *d, uint8_t spr)
{
    uint8_t y, v;
    switch (d->state) {
    case 0:
        gx_spr_hide(spr);
        return;
    case 1:                                     /* decollage */
        d->y -= 2;
        div_move_x(d);
        if (!--d->timer) d->state = 2;
        break;
    case 2:                                     /* attaque */
        v = ply.wave >> 2;
        y = (uint8_t)d->y + (v < 2 ? v : 2) + 2;
        d->y = y;
        if (y >= 200) {                         /* sorti en bas : reapparait en haut */
            d->state = 3;
            d->y = -24;
            break;
        }
        if (!(frame_count & 3)) {               /* guidage vers le vaisseau */
            if (ship_x - d->x < 0) {
                if (d->vx != -4) d->vx--;
            } else if (d->vx != 4) {
                d->vx++;
            }
        }
        div_move_x(d);
        if (!--d->bomb) {                       /* bombe */
            d->bomb = (rand8() & 31) + 30;
            if (!ship_dead && y < 140) spawn_bomb(d->x, y + 8);
        }
        if (!ship_dead && abs16((int16_t)y - SHIP_Y) < 14 && abs16(ship_x - d->x) < 24) {
            kill_ship();                        /* collision avec le vaisseau */
            d->state = 0;
            alien_destroy(d->row, d->col);
            gx_spr_hide(spr);
            return;
        }
        break;
    default:                                    /* 3 : retour vers la formation */
        d->x = fx + d->col * COL_PITCH;
        d->y += 2;
        if (d->y >= (uint8_t)(fy + d->row * ROW_PITCH)) {
            d->state = 0;                       /* arrive : reprend sa place */
            diving[d->row] &= ~bits[d->col];
            gx_spr_hide(spr);
            return;
        }
        break;
    }
    gx_spr_set(spr, d->x, d->y, F_DIVE_A + wave_type, ZOOM_X2Y1);
}
```

7.4 Collisions des tirs

`hit_formation` transforme la position du tir en colonne et rangée par des soustractions successives de 48 et de 24. Une division appellerait une routine de la bibliothèque de SDCC, beaucoup plus lente (le profil le montrait). Le tir doit tomber dans la partie dessinée de l'alien : de 4 à 27 pixels dans la colonne, et sur les 16 premières lignes de la rangée.

```
uint8_t hit_formation(int16_t hx, uint8_t hy)
```

```
{
    int16_t dx = hx - fx;
    uint8_t c, r, rx, ry;
    if (dx < 0 || dx >= 512) return 0;
    for (c = 0; dx >= COL_PITCH; c++) dx -= COL_PITCH;  /* colonne (division sans routine de
division) */
    rx = (uint8_t)dx;
    if (c >= 6 || rx < 4 || rx >= 28) return 0; /* entre deux colonnes */
    if (hy < fy) return 0;
    ry = hy - fy;
    for (r = 0; ry >= ROW_PITCH; r++) ry -= ROW_PITCH;  /* rangee */
    if (ry >= 16 || r >= 4) return 0;
    if (!alien_visible(r, c)) return 0;
    alien_destroy(r, c);
    add_score(row_pts[r]);
    expl_start(fx + c * COL_PITCH, fy + r * ROW_PITCH);  /* explosion a la place de l'alien
*/
    return 1;
}
```

7.5 Bombes et vaisseau mère

Deux bombes au plus (sprites 9 et 10). La formation en lâche une à intervalles qui raccourcissent avec la vague. Une fois sur deux, elle vient de la colonne au-dessus du vaisseau. Les plongeurs et le vaisseau mère (ses deux canons) en lâchent aussi. Le vaisseau mère flotte verticalement, traverse l'écran de bord à bord et accélère sous 12 points de vie. Détruit, il produit 9 explosions en chaîne pendant 72 images avant la fin de la vague.

```
void boss_update(void)
{
    uint8_t a, r;
    if (bflash) bflash--;
    if (bdeath) {                               /* detruit : explosions en chaine */
        if (--bdeath & 7) return;
        bflash = 2;
        r = rand8() & 127;
        if (r >= 100) r -= 40;
        a = rand8() & 31;
        expl_start(bx + r, by + a + 8);
        return;
    }
    if (!ply.bhp || frozen) return;
    a = (frame_count >> 2) & 31;                /* flottement vertical */
    if (a >= 16) a = ~a & 15;
    by = a + 24;
    a = ply.bhp < 12 ? 4 : 2;                   /* blesse : plus rapide */
    if (!bdir) {
        bx += a;
        if (bx >= 640 - 16 - 128) bdir = 1;
    } else {
        bx -= a;
        if (bx < 16) bdir = 0;
    }
    if (ship_dead || --bfire) return;
    bfire = (rand8() & 15) + 18;
    spawn_bomb(bx + ((rand8() & 0x80) ? 80 : 16), by + 52);  /* canon gauche ou droit */
}
```

8. fx.c : hasard, score, panneau, étoiles, explosions

8.1 Le hasard

`rand8` est le générateur « xorshift » 16 bits de la version assembleur, réécrit en C, opération pour opération : les parties suivent les mêmes suites de hasard. Elle est déclarée `__sdcccall(0)`, car stars.asm l'appelle (résultat dans L).

```
uint8_t rand8(void) __sdcccall(0)
{
    uint8_t h = rnd >> 8, l = (uint8_t)rnd;
```

```
    h ^= (uint8_t)((l >> 1) | (h << 7));
    l ^= (uint8_t)((h >> 1) | (l << 7));
    h ^= l;
    rnd = ((uint16_t)h << 8) | l;
    return h;
}
```

8.2 Score, record et panneau

```
void add_score(uint16_t tens)
{
    uint16_t old = ply.score;
    ply.score += tens;
    if (ply.score < old) ply.score = 0xFFFF;        /* (plafond : 655 350 points) */
    hud_dirty |= 1;
    if (old < 1000 && ply.score >= 1000) {
        hud_dirty |= 2;
        ply.lives++;
        GX_SOUND(CAPSULE);
    }
}

/* update_hi : record battu ? (1 = oui, record mis a jour) */
uint8_t update_hi(void)
{
    if (ply.score <= hiscore) return 0;
    hiscore = ply.score;
    return 1;
}
```

Le panneau n'est pas redessiné à chaque image : `hud_dirty` note ce qui a changé (bit 0 : score, bit 1 : vies et vague), et `hud_update` ne réécrit que cela, à la fin de play_frame. Écrire du texte prend du temps (chaque caractère fait 8 lignes de 2 octets).

8.3 Messages

Les messages (« VAGUE 01 », « BRAVO ! »…) s'écrivent dans une bande de 28 lignes (124 à 151). Tant que `star_skip` vaut 1, les étoiles ne la traversent pas, pour ne pas abîmer le texte.

8.4 Les étoiles

40 étoiles de 4 octets (colonne, motif, ligne, vitesse) descendent à 3 vitesses. La vitesse fixe aussi l'encre, donc la luminosité : les plus rapides sont les plus claires, ce qui donne une impression de profondeur. Le motif est un octet du mode 1 qui allume un seul pixel dans l'encre voulue. `stars_init` (en C) les place au hasard ; `stars_update` les fait avancer à chaque image : elle est en assembleur (chapitre 9). La version C est conservée sous `#if STARS_EN_C` pour comparer.

```
void stars_init(void)
{
    uint8_t i, a, sp;
    star_t *s = stars;
    for (i = NSTARS; i; i--, s++) {
        s->col = star_col();
        a = rand8() & 127;
        s->y = a + (rand8() & 31) + 16;
        sp = i & 3;
        if (!sp) sp = 1;
        s->speed = sp;                              /* vitesse 1 a 3 = encre */
        s->pat = pat_tab[(sp - 1) * 4 + (rand8() & 3)];
    }
}
```

8.5 Explosions et perte du vaisseau

```
void expl_update(void)
{
    if (!expl_t) {
        gx_spr_hide(SPR_EXPL);
```

```
        return;
    }
    expl_t--;
    gx_spr_set(SPR_EXPL, expl_x, expl_y, F_EXPL1 + ((uint8_t)(~expl_t & 15) >> 2),
ZOOM_X2Y1);
}
```

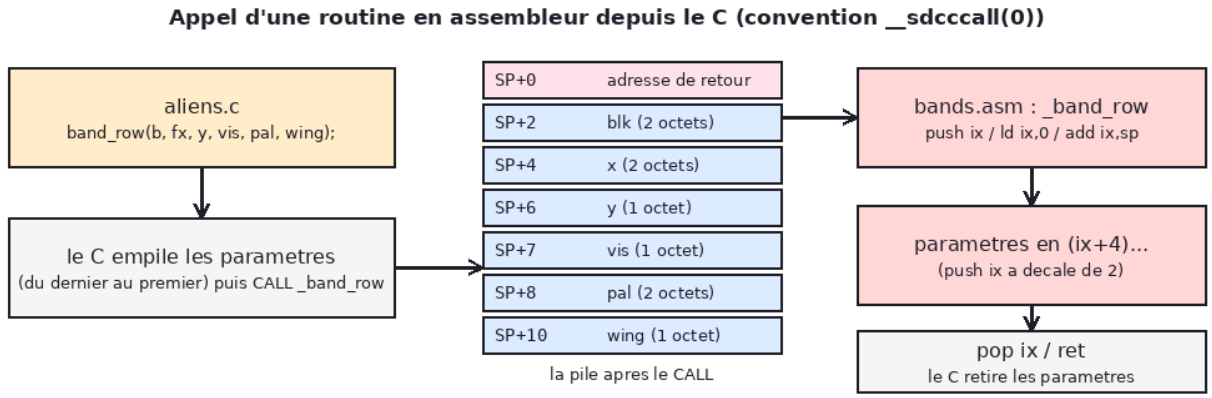
```
void kill_ship(void)
{
#if GOD_MODE
    return;
#else
    if (ship_dead) return;
    ship_dead = 120;
    expl_quiet(ship_x, SHIP_Y);
    gx_spr_hide(SPR_SHIP);
    march_stop();
    GX_SOUND(MORT);
    ply.power = 0;
#endif
}
```

9. Les routines en assembleur

Deux routines sont écrites en assembleur Z80 dans src/, incluses par main.asm à côté de la liaison lib/c/gx_c.asm, et déclarées dans stellar.h comme des fonctions C ordinaires :

```
void stars_update(void) __sdcccall(0);                          /* src/stars.asm */
void band_row(uint8_t *blk, int16_t x, uint8_t y, uint8_t vis,
              const uint16_t *pal, uint8_t wing) __sdcccall(0); /* src/bands.asm */
```

9.1 La convention d'appel



band_row : les paramètres sur la pile, lus par IX

- En assembleur, le nom C prend un _ devant : `_band_row`, `_stars`, `_star_skip`, `_rand8`.
- Avec `__sdcccall(0)`, les paramètres sont **sur la pile**, à partir de SP+2 : un octet pour un uint8_t, deux pour un int16_t ou un pointeur, dans l'ordre de la déclaration. C'est l'appelant qui les retire après le retour.
- Le résultat revient dans **L** (8 bits) ou **HL** (16 bits) : rand8 rend son nombre dans L.
- L'assembleur peut modifier A, B, C, D, E, H, L et les drapeaux. Il doit **conserver IX**, le pointeur de pile du C. IY n'est pas utilisé par le C.

9.2 bands.asm : une bande de sprites

band_row remplit le bloc de 38 octets d'une rangée : six fois X, Y et zoom, puis les quatre couleurs. Le zoom vaut 0 pour un alien absent, ce qui cache le sprite. Une des couleurs passe ensuite au noir pour le battement d'ailes. La routine sauve IX, puis le place sur la pile pour lire ses

paramètres en (ix+4)…, comme les fonctions de lib/c/gx_c.asm.

```
_band_row:
        push ix
        ld ix,0
        add ix,sp                       ; (ix+4) = premier parametre
        ld l,(ix+4)
        ld h,(ix+5)                     ; HL = bloc
        ld e,(ix+6)
        ld d,(ix+7)                     ; DE = X
        ld c,(ix+9)                     ; C = aliens visibles
        ld b,6
bdr_spr:
        ld (hl),e                       ; X
        inc hl
        ld (hl),d
        inc hl
        ld a,(ix+8)                     ; Y
        ld (hl),a
        inc hl
        ld (hl),0
        inc hl
        xor a
        rr c
        jr nc,bdr_z
        ld a,ZOOM_X2Y1
bdr_z:  ld (hl),a                       ; zoom (0 = invisible)
        inc hl
        ld a,e                          ; colonne suivante : X + 48
        add a,48
        ld e,a
        jr nc,bdr_c
        inc d
bdr_c:  djnz bdr_spr
```

9.3 stars.asm : les étoiles

stars_update parcourt le tableau stars[] (défini en C dans fx.c) avec HL. Pour chaque étoile, elle efface l'ancien point, ajoute la vitesse à la ligne et redessine. Une étoile arrivée en bas repart en haut, dans une colonne choisie par la fonction C `rand8`. L'adresse à l'écran se calcule avec la table **line_addr** générée par le studio. En mode 1, les lignes ne se suivent pas en mémoire : la ligne y+1 est 2 Ko plus loin que la ligne y, sauf toutes les 8 lignes.

```
_stars_update:
        ld hl,_stars                    ; HL parcourt le tableau (IX n'est pas utilise)
        ld b,ST_NB
st_loop:
        push bc
        ld c,(hl)                       ; C = colonne
        inc hl
        inc hl
        ld a,(hl)                       ; A = ligne
        call st_ptr                     ; DE = adresse de l'etoile a l'ecran
        jr z,st_1
        xor a
        ld (de),a                       ; efface
st_1:   ld a,(hl)
        inc hl
        add a,(hl)                      ; descend de sa vitesse
        dec hl
        cp 184
        jr c,st_2
        sub 168                         ; repart en haut...
        ld (hl),a
        dec hl
        dec hl
        push hl
        call st_col                     ; ...dans une colonne au hasard
        pop hl
        ld (hl),a
        ld c,a
        inc hl
        inc hl
```

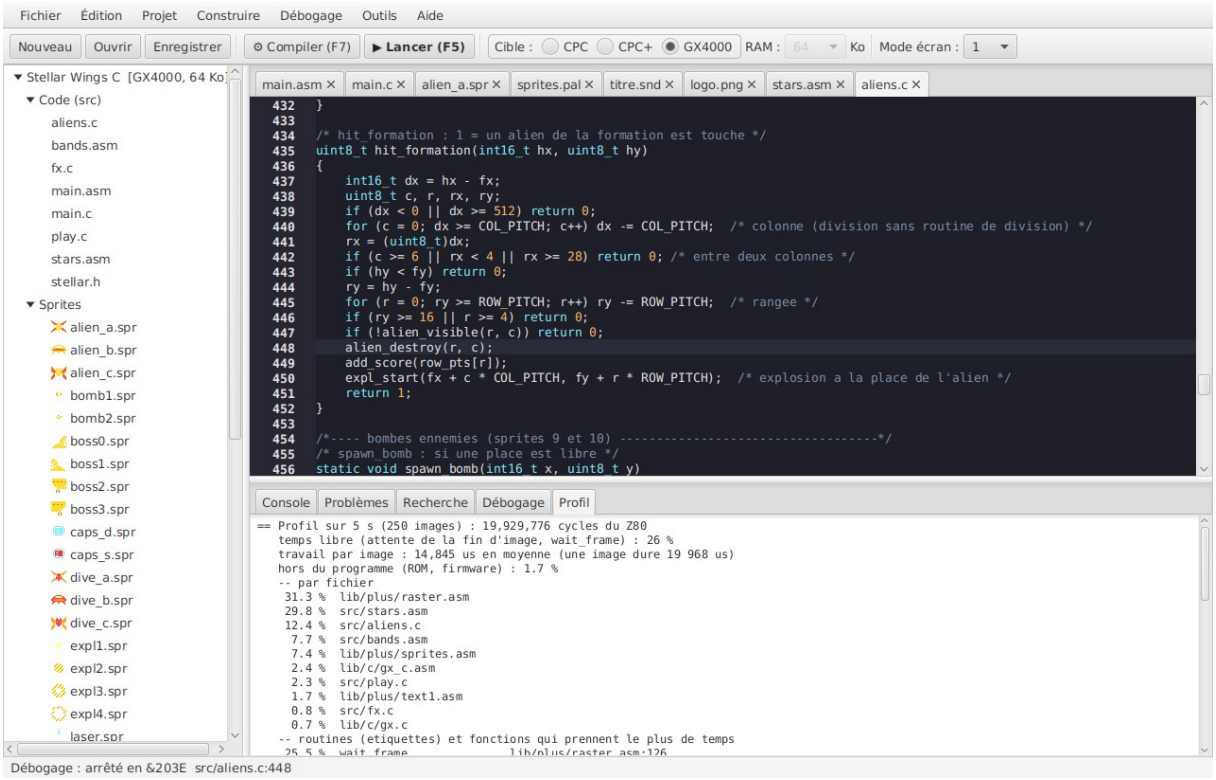
```
        ld a,(hl)
st_2:   ld (hl),a
        call st_ptr
        jr z,st_3
        dec hl
        ld a,(hl)                       ; motif
        ld (de),a                       ; dessine
        inc hl
st_3:   inc hl                          ; etoile suivante
        inc hl
        pop bc
        djnz st_loop
        ret
```

```
; st_ptr : A = ligne, C = colonne -> DE = adresse ecran (NZ) ;
;          Z = ligne dans la zone des messages (ne rien toucher). HL est preserve.
st_ptr:
        ld e,a
        ld a,(_star_skip)
        or a
        jr z,stp_ok
        ld a,e
        sub ST_MSG_TOP
        cp ST_MSG_LINES
        jr nc,stp_ok
        xor a                           ; Z
        ret
stp_ok: push hl
        ld l,e
        ld h,0
        add hl,hl
        ld de,line_addr                 ; table generee par le studio (build/gen.asm)
        add hl,de
        ld a,(hl)
        inc hl
        ld d,(hl)
        add a,c                         ; + colonne
        ld e,a
        jr nc,stp_1
        inc d
stp_1:  pop hl
        or 1                            ; NZ (D vaut au moins &C0)
        ret
```

10. Performances

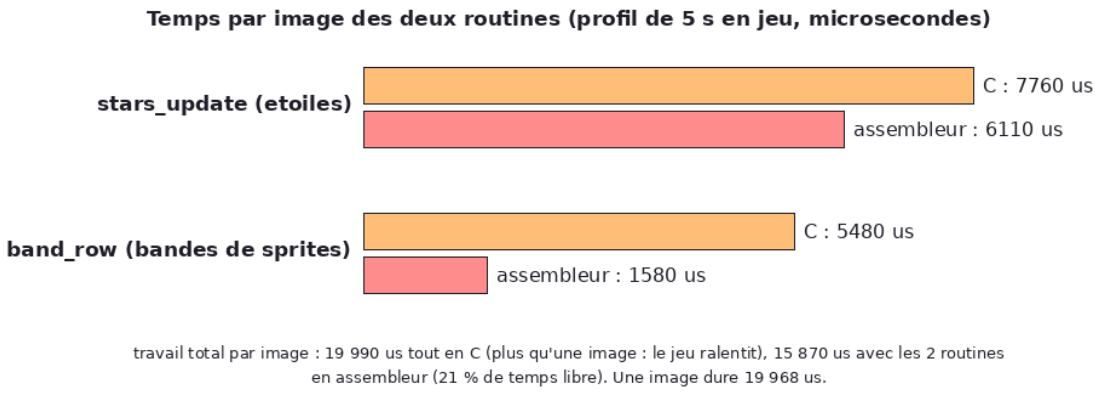
Une image dure 19 968 microsecondes (µs). Pour tourner à 50 images par seconde, le jeu doit tout faire dans ce temps : la logique, l'affichage du texte, les interruptions raster et la recopie des sprites. Sinon `gx_wait_frame` arrive après la fin de l'image et attend la suivante : le jeu ralentit de moitié par moments.

La première version de STELLAR WINGS C était **entièrement en C**. Elle ralentissait. **Débogage > Mesurer les performances** (Ctrl+F6) a donné le profil ci-dessous. Le studio compte pendant 5 secondes les cycles du Z80 passés à chaque adresse (Caprice32 corrigé 0.5.1), puis les regroupe par fichier, par fonction et par ligne grâce aux symboles de la compilation.



L'onglet Profil après 5 secondes de jeu (version actuelle : C + 2 routines en assembleur)

Mesure (5 s de jeu, 250 images)	Tout en C	C + stars.asm + bands.asm
Travail par image (moyenne)	19 990 µs : plus d'une image	15 870 µs
Temps libre (attente de la fin d'image)	0 %	21 %
Étoiles (stars_update + star_ptr)	38,8 % = 7 760 µs par image	30,5 % = 6 110 µs
Bandes de sprites (band_row, put_spr, put_colors)	27,4 % = 5 480 µs par image	7,9 % = 1 580 µs
Résultat	le jeu saute des images (environ 1 sur 10)	50 images par seconde



Les deux routines, en C puis en assembleur

Ce qu'on en retient :

- **Mesurer avant d'optimiser.** Deux fonctions prenaient les deux tiers du temps ; tout le reste du jeu (collisions, plongeurs, vaisseau mère, panneau) ne coûte presque rien. Réécrire le jeu entier en assembleur n'aurait rien apporté de plus.
- **band_row** gagne beaucoup (3,5 fois plus rapide) : en C, chaque écriture dans le bloc passe par un pointeur rangé dans la pile (accès par IX), alors qu'en assembleur HL avance tout seul (`ld (hl),e : inc hl`).

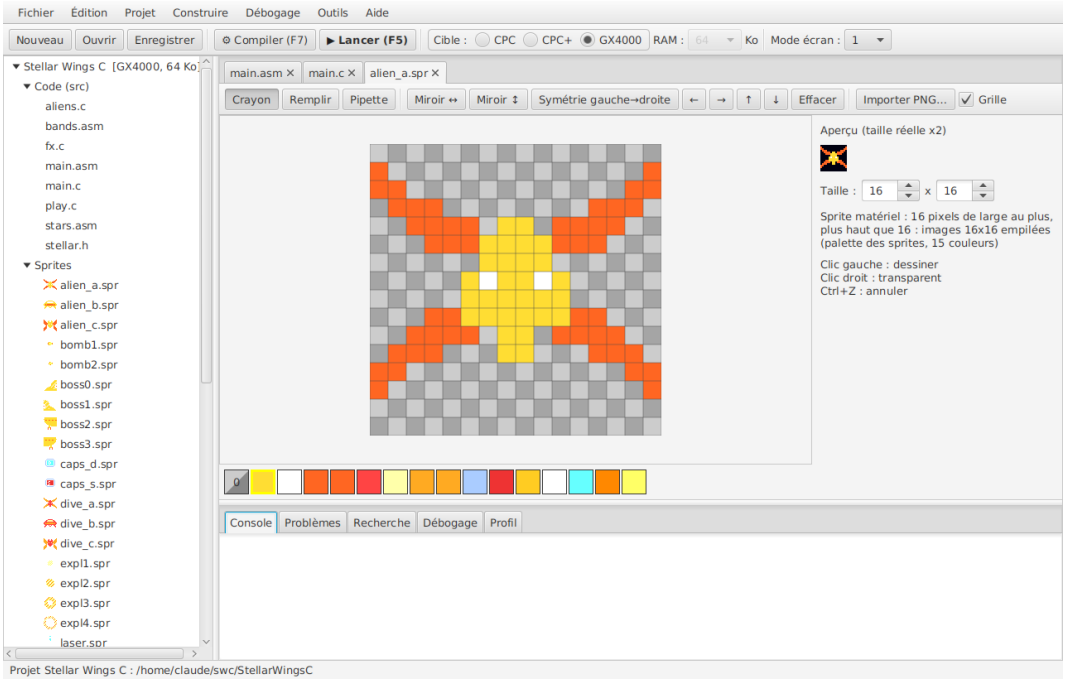
- **stars_update** gagne peu (1,3 fois) : le temps passe dans le calcul de l'adresse à l'écran (lecture de line_addr), qui coûte presque autant en assembleur. Pour aller plus vite, il faudrait changer d'algorithme, par exemple garder dans chaque étoile l'adresse où elle a été dessinée pour l'effacer sans recalcul. C'est un bon exercice, en C comme en assembleur.
- Les divisions (/ 48, % 24) appelaient une routine de SDCC qui apparaissait aussi dans le profil : de simples boucles de soustraction les remplacent (hit_formation, bomb_drop).

Pour refaire la comparaison : **Projet > Propriétés**, définitions `STARS_EN_C=1 BANDES_EN_C=1`. Les versions C des deux routines sont alors compilées à la place des fichiers .asm (`#if` dans fx.c et aliens.c, `ifndef` dans main.asm). Ensuite F5, une partie, puis Ctrl+F6.

11. Les ressources

Toutes les ressources sont des fichiers du dossier assets/. On les modifie dans les éditeurs du studio, et F5 les reconvertit. Elles ont été reprises de la version assembleur à l'identique. Après conversion, sprites, palettes, logo et listes de sons sont les mêmes octet pour octet, ou donnent les mêmes écritures dans le PSG aux mêmes instants (vérifié).

11.1 Sprites

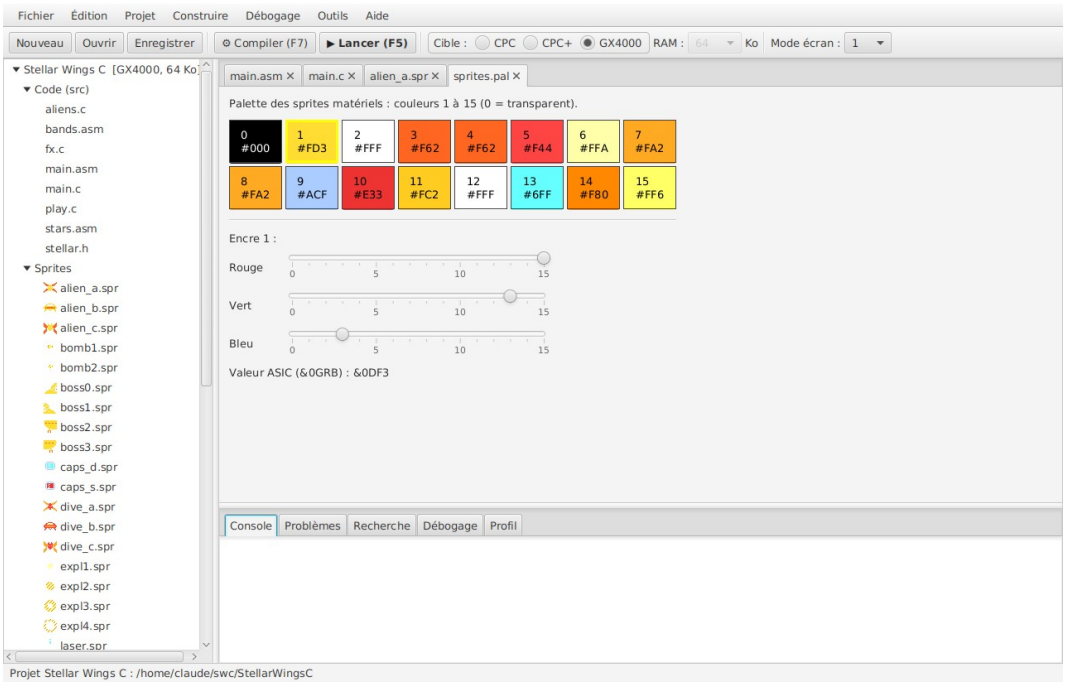


alien_a.spr dans l'éditeur de sprites

Fichiers	Images	Utilisation
alien_a, alien_b, alien_c	F_ALIEN_A + type	formation (type = (vague - 1) modulo 3)
dive_a, dive_b, dive_c	F_DIVE_A + type	plongeurs : les mêmes dessins en couleurs 5-8
ship, laser	F_SHIP, F_LASER	vaisseau, tir
bomb1, bomb2	F_BOMB1 + (image / 4) modulo 2	bombe animée
expl1 à expl4	F_EXPL1 + étape	explosion en 4 images (16 images d'affichage)
caps_d, caps_s	F_CAPS_D + type	capsules
boss0 à boss3	F_BOSS0 + quart	vaisseau mère (4 quarts)

Le studio numérote les images dans l'ordre alphabétique des fichiers. Le programme utilise des calculs comme F_ALIEN_A + type : les fichiers d'une même famille doivent donc rester consécutifs (alien_a, alien_b, alien_c).

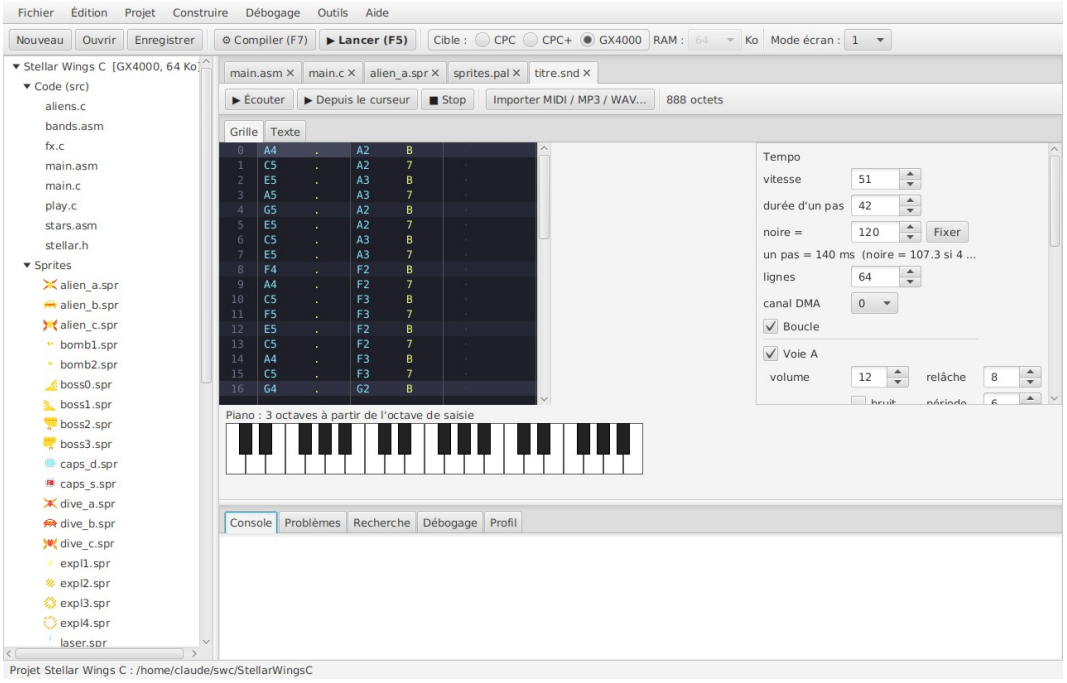
11.2 Palettes



sprites.pal : couleurs 1 à 15 des sprites (&0GRB sous les curseurs)

Les couleurs 1 à 4 des sprites sont changées à chaque rangée par les bandes raster (rowpal_tab dans aliens.c) : les aliens de la formation n'utilisent que ces 4 couleurs. Les plongeurs utilisent les couleurs 5 à 8. Les couleurs 7 et 8 font battre les ailes (play_frame). L'écran utilise ecran.pal en jeu et pal_title (dans main.c) sur l'écran titre.

11.3 Sons

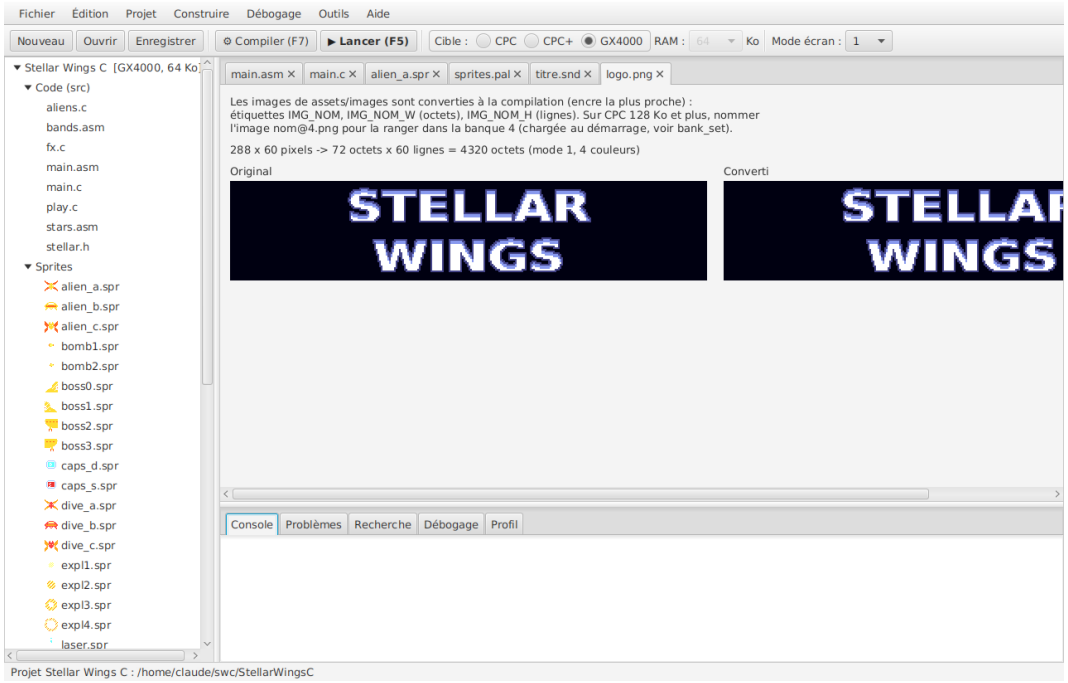


titre.snd : musique écrite en notes (voies A et B)

Fichier	Canal / vitesse	Utilisation

titre.snd	0 / 51	musique de l'écran titre (mélodie et basse, en boucle)
marche.snd	0 / variable	marche de la formation, en boucle ; tempo = aliens restants + 4 (5 pour le vaisseau mère)
vague.snd, bravo.snd, gameover.snd	0 / 51	jingles
tir.snd, capsule.snd, boss_touche.snd	1 / 0	bruitages sur la voie B
boom.snd, mort.snd	2 / 0	explosions (bruit sur la voie C)

11.4 Le logo



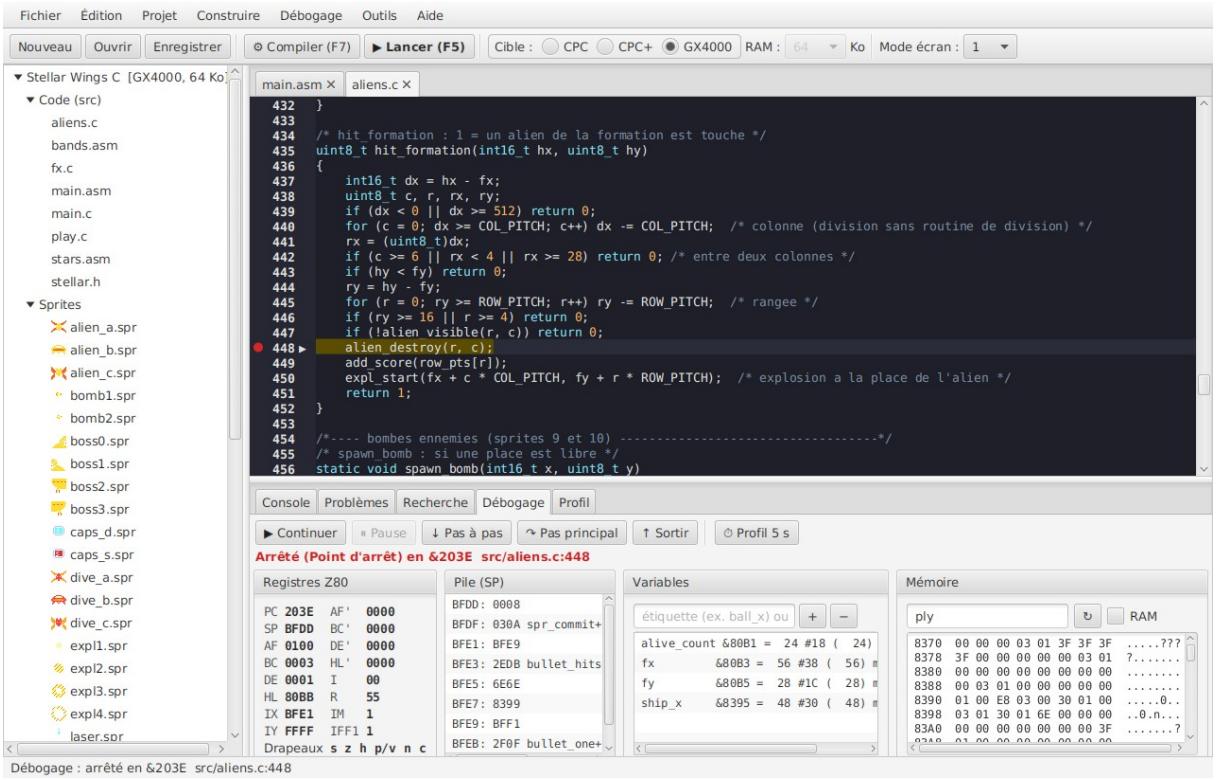
logo.png et sa conversion en mode 1

logo.png (288 x 60 pixels) utilise les 4 couleurs de ecran.pal. Le studio le convertit en octets mode 1 (IMG_LOGO, 72 x 60 octets, page 2). Sur l'écran titre, il est affiché avec la palette pal_title. L'encre 3 change toutes les 4 lignes (rasters), ce qui donne le dégradé.

12. Mise au point

Définition (Projet > Propriétés)	Effet
START_WAVE=4	commence à la vague 4 (vaisseau mère)
GOD_MODE=1	vaisseau invincible
STARS_EN_C=1, BANDES_EN_C=1	versions C des deux routines assembleur (comparaison, chapitre 10)

Les définitions sont passées à SDCC et à rasm. Dans le code, elles sont entourées de `#ifndef` (stellar.h) : la valeur par défaut ne s'applique que si rien n'est défini. Penser à vider le champ avant de faire la cartouche finale.



Point d'arrêt sur alien_destroy(r, c) dans hit_formation : variables C et mémoire du joueur (ply)

- **F9** sur une ligne C pose un point d'arrêt ; **F5** lance. L'exécution s'arrête quand un tir touche un alien.
- **F11** avance d'une ligne C (en entrant dans alien_destroy), **F10** passe l'appel, **Maj+F11** termine la fonction, **F8** continue.
- Dans **Variables**, on tape un nom de variable globale ou static : alive_count, fx, fy, ship_x… Les variables locales (r, c) ne sont pas visibles : SDCC les garde dans des registres ou la pile.
- Dans **Mémoire**, `ply` montre le joueur en cours : score (2 octets), vies, vague, les 4 octets alive, power, over, bhp.
- La **pile** affiche les fonctions appelantes : hit_formation, bullet_hits, bullet_one, play_frame…

Le dossier tools/ du projet n'est pas nécessaire au studio. La variable dbg_loops (play.c) compte les images de jeu. Comparée à frame_count, elle montre si le jeu saute des images.

13. Comparaison avec la version assembleur

	Assembleur (STELLAR WINGS)	C (STELLAR WINGS C)
Source du jeu	game/*.asm : 2 760 lignes	src/*.c et stellar.h : 1 370 lignes + 2 routines asm (170 lignes)
Code (page 0, bibliothèque comprise)	8,5 Ko	12,5 Ko (+ 2,6 Ko d'écran de présentation)
Variables du jeu	VAR dans vars.asm, tables en &8800	c_ram : 764 octets
Temps libre par image (profil, en jeu)	35 %	20 à 30 % selon les moments
Score	BCD sur 3 octets (daa)	entier en dizaines de points
Ressources	produites par un script Python	ressources du studio (éditeurs)
Bibliothèque	celle de 2026 (ligne d'armement fixe)	GX Studio 0.5.1 : ligne d'armement mesurée au démarrage (pri_detect), marche sur toutes

		les machines

Le comportement du jeu est le même. Les règles, les vitesses, les collisions et le hasard sont recopiés opération pour opération. Un seul point diffère : le score plafonne à 655 350 points.

Faire évoluer le jeu

- **Nouvelle forme d'alien** : un fichier alien_d.spr (et dive_d.spr), puis `% 4` au lieu de `% 3` dans wave_prepare, et 16 couleurs de plus dans rowpal_tab.
- **Plus de rangées** : la table raster a de la place (80 octets) ; il faut agrandir bands[] (38 octets par rangée) et vérifier le temps libre avec le profil.
- **Étoiles plus rapides** : garder l'adresse de dessin dans chaque étoile (chapitre 10).
- **Version CPC** : la bibliothèque lib/cpc remplace les sprites matériels par des sprites logiciels ; le multiplexage et les rasters n'existent pas, la formation devrait être dessinée dans l'écran.

Annexe : index des fonctions

Fonction	Fichier	Rôle
main	main.c	boucle titre / partie
screen_reset	main.c	tout couper, effacer
title_screen, title_raster, title_parade	main.c	écran titre
game, player_init, game_over, player_over_msg	main.c	partie, joueurs
play_turn, play_frames, next_wave	main.c	tour de jeu, vagues
msg_player, msg_wave	main.c	messages
play_frame	play.c	une image de jeu
turn_reset	play.c	positions de départ
ship_update, fire_bullet	play.c	vaisseau, tir
bullets_update, bullet_one, bullet_hits	play.c	tirs du joueur
caps_update, spawn_caps	play.c	capsule
aliens_reset, wave_prepare, wave_cleared	aliens.c	vagues
count_alive, alien_visible, alien_destroy, col_bottom	aliens.c	état de la formation
form_update, march_start, march_stop, march_tempo_val	aliens.c	mouvement, marche
next_tab, tab_tail, build_raster, build_boss, put_spr, put_colors	aliens.c	tables raster
div_launch, diver_one, div_move_x, divers_update	aliens.c	plongeurs
hit_divers, hit_formation, hit_boss	aliens.c	collisions des tirs
bomb_drop, spawn_bomb, bomb_one, bombs_update	aliens.c	bombes
boss_update	aliens.c	vaisseau mère
rand8, abs16	fx.c	hasard, valeur absolue
add_score, update_hi, print_score	fx.c	score, record
hud_draw, hud_update, hud_lives	fx.c	panneau
msg_open, msg_close, msg_center	fx.c	zone des messages
stars_init, star_col	fx.c	étoiles (placement)

expl_start, expl_quiet, expl_update, expl_active, expl_reset	fx.c	explosion
kill_ship	fx.c	vaisseau touché
_stars_update	stars.asm	étoiles (chaque image)
_band_row	bands.asm	bloc d'une bande de sprites